计算机系统 I

第十章：计算器

数据类型转换

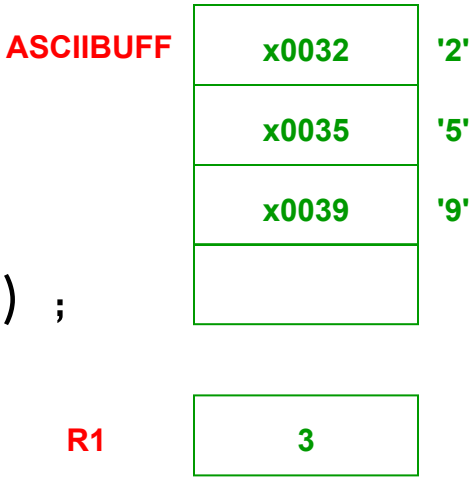
- 计算机程序，键盘输入数字，显示器输出计算结果。
- 考虑如下程序：

```
TRAP  x23        ; input from keybd
ADD   R1, R0, #0  ; move to R1
TRAP  x23        ; input from keybd
ADD   R0, R1, R0  ; add two inputs
TRAP  x21        ; display result
TRAP  x25        ; HALT
```

- 输入 2 和 3 – 发生什么？
- 屏幕显示结果： e
- 为什么？ ASCII '2' (x32) + ASCII '3' (x33) = ASCII 'e' (x65)

ASCII 到 二进制

- 在处理多位数的数字时很有用
- 假设读了3位ASCII数到内存缓冲
- 如何将其转换为ASCII码？
  - 将第一位字符转换为数字并乘以100（200）；
  - 将第二位字符转换为数字并乘以10（50）；
  - 将第三位字符转换为数字（9）.
  - 将三个数相加（259）



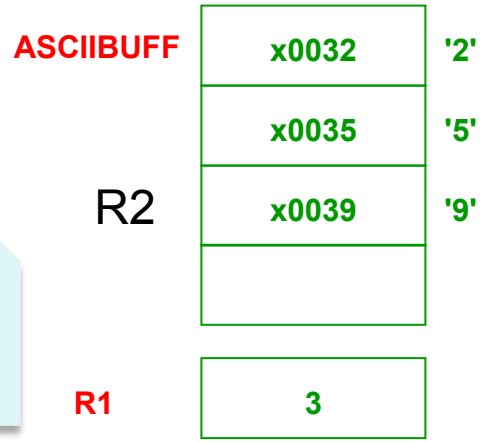
ASCII码到二进制的转换程序（1）

```
; Three-digit buffer at ASCIIBUF.
; R1 tells how many digits to convert.
; Put resulting decimal number in R0.
ASCIItoBinary
        ST    R1, AToB_Save1
        ST    R2, AToB_Save2
        ST    R3, AToB_Save3
        ST    R4, AToB_Save4
        AND   R0, R0, #0  ; clear result
        ADD   R1, R1, #0  ; test # digits
        BRz   AtoB_Done   ; done if no digits
;
        LD    R2, AtoB_ASCIIBUFF  ; R2 points to ASCIIBUFF
        ADD   R2, R2, R1
        ADD   R2, R2, #-1 ; points to ones digit
```

存储即将使用的通用寄存器当前的内容

初始化寄存器

R2指向个位数的ASCII码所在地址



ASCII码到二进制的转换程序（2）

```
        LDR  R4, R2, #0  ; load “one” digit
        AND  R4, R4, x000F  ; convert ASCII to number
        ADD  R0, R0, R4  ; add ones contrib
;
        ADD   R1, R1, #-1  ; one less digit
        BRz   AtoB_Done     ; done if zero
        ADD  R2, R2, #-1  ; points to tens digit
;
        LDR   R4, R2, #0   ; load digit
        AND  R4, R4, x000F  ; convert ASCII to number
        LEA   R3, LookUp10 ; multiply by 10
        ADD  R3, R3, R4
        LDR   R4, R3, #0
        ADD  R0, R0, R4    ; adds tens contrib
```

将个位数加载至R4进行转换

将R2指针移向十位数所在地址

将十位数加载至R4，并进行转换，利用十位表进行查找

ASCII码到二进制的转换程序（3）

```
            ADD  R1, R1, #-1   ; one less digit
            BRz   AtoB_Done     ; done if zero
            ADD  R2, R2, #-1   ; points to hundreds digit
;
            LDR   R4, R2, #0   ; load digit
            AND  R4, R4, x000F  ; convert ASCII to number
            LEA   R3, LookUp100 ; multiply by 100
            ADD  R3, R3, R4
            LDR   R4, R3, #0
            ADD  R0, R0, R4   ; adds 100‘s contrib
;
AtoB_Done   LD R1, AtoB_Save1
            LD R2, AtoB_Save2
            LD R3, AtoB_Save3
            LD R4, AtoB_Save4
             RET
```

将R2指针移向百位数所在地址

将百位数加载至R4，并进行转换，
利用百位表进行查找

ASCII码到二进制的转换程序（4）

```
AtoB_ASCIIBUF     .FILL  ASCIIBUF
AtoB_Save1        .BLKW #1
AtoB_Save2        .BLKW #1
AtoB_Save3        .BLKW #1
AtoB_Save4        .BLKW #1
LookUp10          .FILL 0
                  .FILL 10
                  .FILL 20
                …
LookUp100         .FILL 0
                  .FILL 100
                …
```

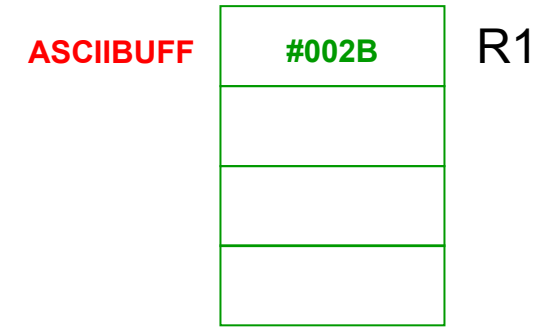
二进制到ASCII码转换

- 将补码转换为包含4位数的ASCII码（包含符号位）
- 这里需要除以100来得到百位数
  - **这里为什么不能使用查找表?**
  - **重复地减100来做除法**
- 转换开始前，需要判断该数的符号
  - **将符号字符 (+ or –) 写入缓存，并将剩下的数变位正数**

二进制到ASCII码的转换程序 (1)

```
; R0 is between -999 and +999.
; Put sign character in ASCIIBUF, followed by three
; ASCII digit characters.

BinaryToASCII
        ST    R0, BToA_Save0
        ST    R1, BToA_Save1
        ST    R2, BToA_Save2
        ST    R3, BToA_Save3
        LD    R1, BtoA_ASCIIBUFF ; pt to result string
        ADD R0, R0, #0    ; test sign of value
        BRn  NegSign
        LD    R2, ASCIIplus ; store '+'
        STR  R2, R1, #0
        BRnzp  Begin100
```



存储即将使用的通用寄存器当前的内容

存储数据符号

二进制到ASCII码的转换程序（2）

```
NegSign LD  R2, ASCIIminus  ; store '-'
        STR R2, R1, #0
        NOT R0, R0          ; convert value to pos
        ADD R0, R0, #1
;
Begin100 LD  R2, ASCIIoffset
         LD  R3, Neg100
Loop100 ADD R0, R0, R3
        BRn End100
        ADD R2, R2, #1   ; add one to digit
        BRnzp  Loop100
End100  STR   R2, R1, #1  ; store ASCII 100's digit
        LD    R3, Pos100
        ADD R0, R0, R3   ; restore last subtract
```

如果是负数，转换成正数

ASCIIBUFF

#002B
#0032
#0035

当前数据减100直至为负数

存储百位数的ASCII码

二进制到ASCII码的转换程序（3）

```
            LD  R2, ASCIIoffset
Loop10  ADD R0, R0, #-10
            BRn End10
            ADD R2, R2, #1  ; add one to digit
            BRnzp  Loop10
;
End10     STR R2, R1, #2  ; store ASCII 10's digit
            ADD R0, R0, #10 ; restore last subtract
Begin1    LD  R2, ASCIIoffset
            ADD R2, R2, R0  ; convert one's digit
            STR R2, R1, #3  ; store one's digit
           LD R0, BtoA_Save0
           LD R1, BtoA_Save1
           LD R2, BtoA_Save2
           LD R3, BtoA_Save3
           RET
```

当前数据减10直至为负数

存储十和个位数的ASCII码

ASCIIBUFF	#002B
	#0032
	#0035
	#0039

二进制到ASCII码的转换程序（4）

```
ASCIIplus          .FILL x002B    ; plus sign
ASCIIneg           .FILL x002D    ; neg sign
ASCIIoffset        .FILL x0030    ; zero
Neg100             .FILL #-100 ; -100
Pos100             .FILL #100
BtoA_Save0         .BLKW #1
BtoA_Save1         .BLKW #1
BtoA_Save2         .BLKW #1
BtoA_Save3         .BLKW #1
BtoA_ASCIIBUFF     .FILL    ASCIIBUFF
```

基于栈的算术运算

- 有些ISA使用栈来代替寄存器完成算术运算：
  - **零地址机，ADD**
  - **该指令从栈弹出两个数并相加，把计算结果压入栈。**

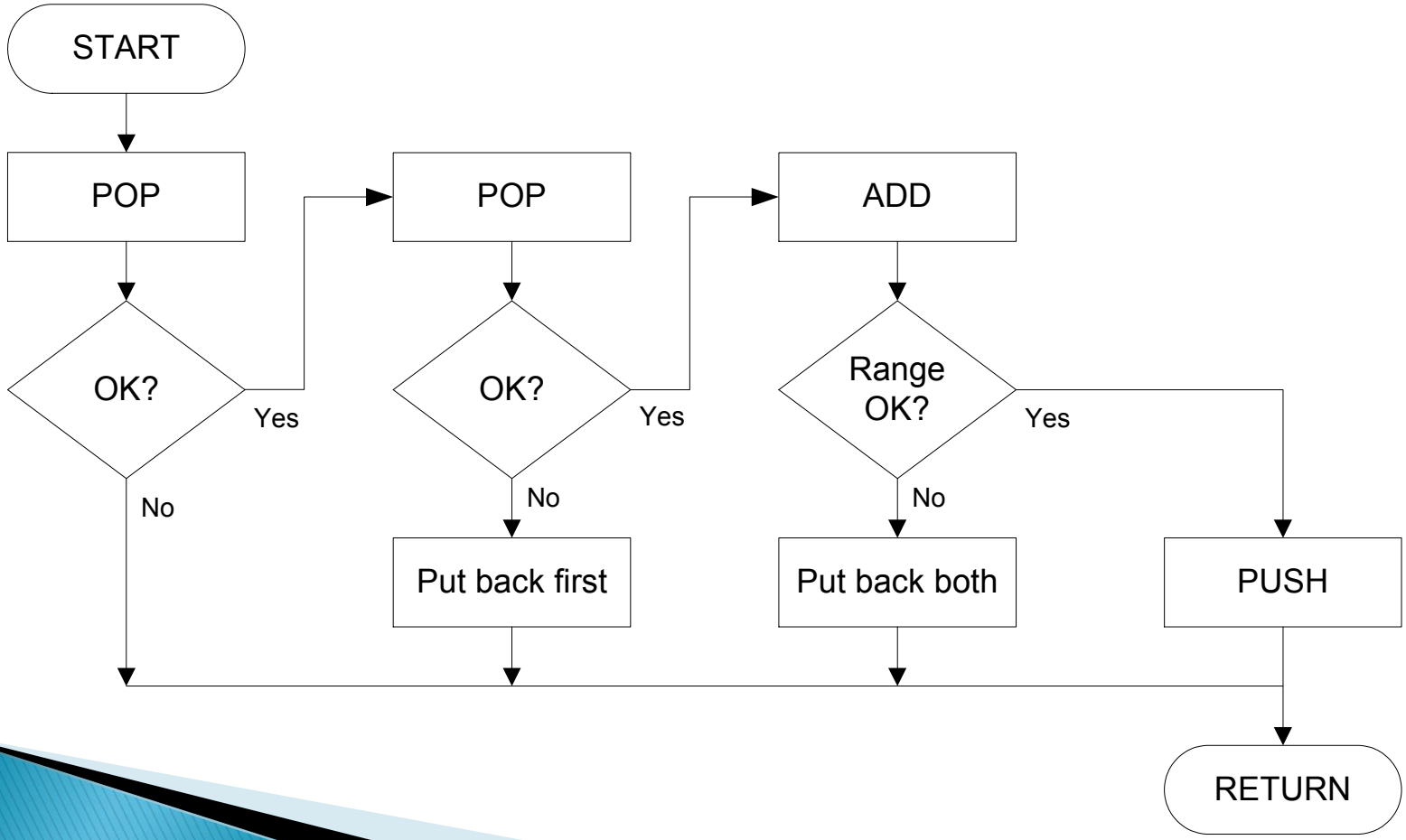
- 例：算术表达式，用栈来计算 (A+B)·(C+D)：

(1) push A
(2) push B
(3) ADD
(4) push C
(5) push D
(6) ADD
(7) MULTIPLY
(8) pop result

为什么使用栈?
- 寄存器个数有限
- 方便子程序调用
- 算法自然地用FILO数据结构表达

基于栈的加法运算流程图

- 从栈里弹出两个值，相加，然后将计算结果压入栈。



基于栈的加法运算程序(1)

```
OpAdd   ST    R0, OpAdd_Save0
        ST    R1, OpAdd_Save1
        ST    R5, OpAdd_Save5
        ST    R7, OpAdd_Save7
        JSR POP             ; Get first operand.
        ADD R5,R5,#0        ; Check for POP success.
        BRp OpAdd_Exit      ; If error, bail.
        ADD R1,R0,#0        ; Make room for second.
        JSR POP             ; Get second operand.
        ADD R5,R5,#0        ; Check for POP success.
        BRp OpAdd_Restore1          ; If err, restore & bail.
        ADD R0,R0,R1        ; Compute sum.
        JSR RangeCheck      ; Check size.
        BRp OpAdd_Restore2          ; If err, restore & bail.
        JSR PUSH            ; Push sum onto stack.
        BRnzp  OpAdd_Exit   ; On to the next task
```

检查是否pop成功

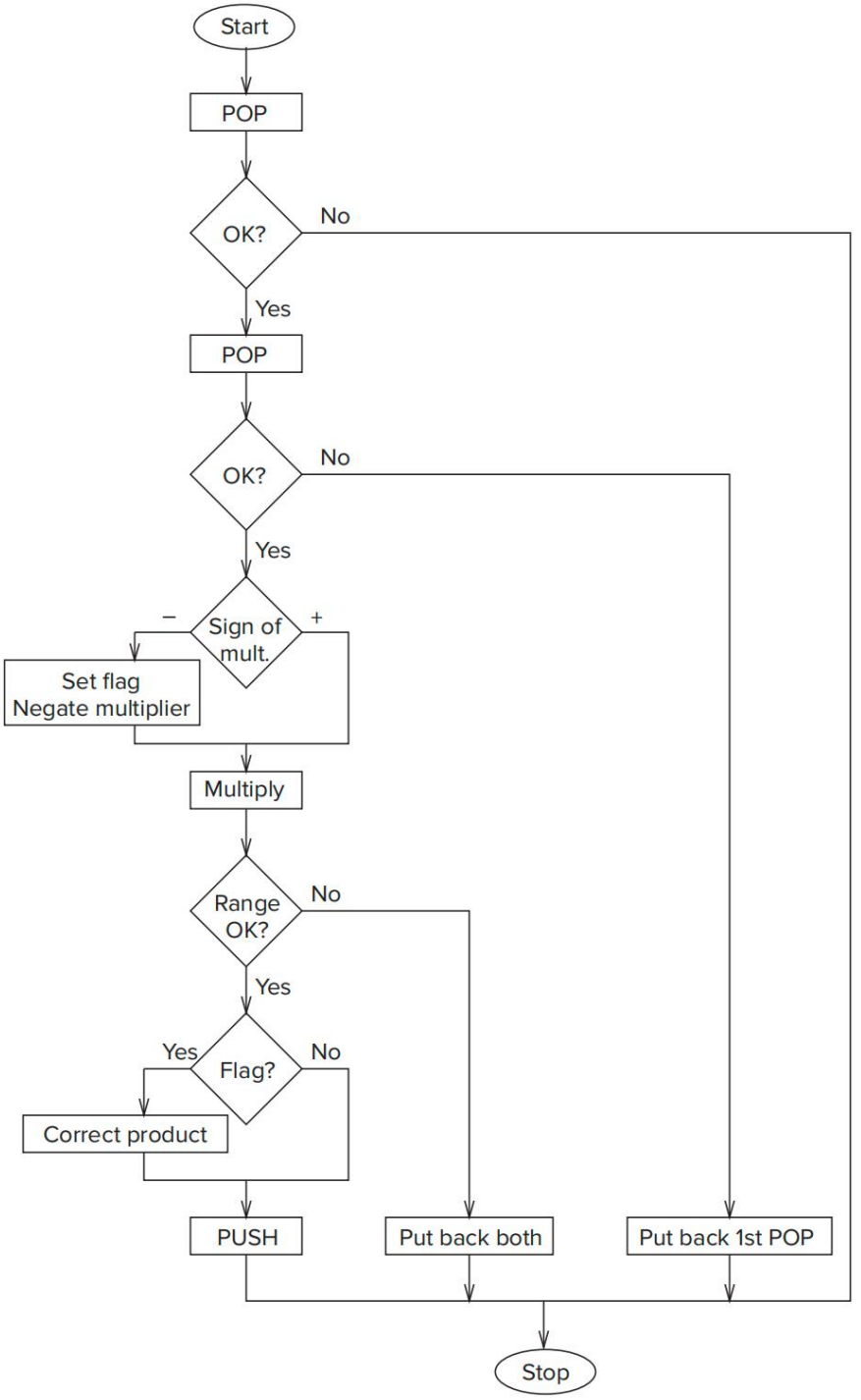
检查相加结果是否溢出

基于栈的加法运算程序(2)

```
OpAdd_Restore2   ADD R6,R6,#-1     ; Decr stack ptr (undo POP)
OpAdd_Restore1   ADD R6,R6,#-1     ; Decr stack ptr
OpAdd_Exit       LD R0, OpAdd_Save0
                 LD R1, OpAdd_Save1
                 LD R5, OpAdd_Save5
                 LD R7, OpAdd_Save7
                 RET
OpAdd_Save0      .BLKW   #1
OpAdd_Save1      .BLKW   #1
OpAdd_Save5      .BLKW   #1
OpAdd_Save7      .BLKW   #1
```

基于栈的乘法运算流程图

- 从栈里弹出两个值，相乘，然后将计算结果压入栈。



基于栈的乘法运算程序(1)

```
; Two values are popped from the stack, multiplied, and if
; their product is within the acceptable range, the result
; is pushed onto the stack. R6 is the stack pointer.
;
OpMult  ST R0,OpMult_Save0
        ST R1,OpMult_Save1
        ST R2,OpMult_Save2
        ST R3,OpMult_Save3
        ST R5,OpMult_Save5
        ST R7,OpMult_Save7
        AND R3,R3,#0 ; R3 holds sign of multiplier.
        JSR POP ; Get first source from stack.
        ADD R5,R5,#0 ; Test for successful POP.
        BRp OpMult_Exit ; Failure
        ADD R1,R0,#0 ; Make room for next POP.
```

检查是否pop成功

基于栈的乘法运算程序(2)

```
            ADD R1,R0,#0 ; Make room for next POP.
            JSR POP ; Get second source operand.
            ADD R5,R5,#0 ; Test for successful POP.
            BRp OpMult_Restore1 ; Failure; restore first POP.
            ADD R2,R0,#0 ; Moves multiplier, tests sign.
            BRzp PosMultiplier
            ADD R3,R3,#1 ; Sets FLAG: Multiplier is neg.
            NOT R2,R2
            ADD R2,R2,#1 ; R2 contains –(multiplier).
PosMultiplier AND R0,R0,#0 ; Clear product register.
            ADD R2,R2,#0
            BRz PushMult ; Multiplier = 0, Done.
;
MultLoop ADD R0,R0,R1 ; THE actual "multiply"
            ADD R2,R2,#–1 ; Iteration Control
```

检查是否负数

利用加法实现乘法

基于栈的乘法运算程序(3)

```
        BRp MultLoop
;
        JSR RangeCheck
        ADD R5,R5,#0 ; R5 contains success/failure.
        BRp OpMult_Restore2
;
        ADD R3,R3,#0 ; Test for negative multiplier.
        BRz PushMult
        NOT R0,R0 ; Adjust for
        ADD R0,R0,#1 ; sign of result.
PushMult JSR PUSH ; Push product on the stack.
        BRnzp OpMult_Exit
OpMult_Restore2 ADD R6,R6,#-1 ; Adjust stack pointer.
OpMult_Restore1 ADD R6,R6,#-1 ; Adjust stack pointer.
OpMult_Exit LD R0,OpMult_Save0
```

检查结果以否溢出

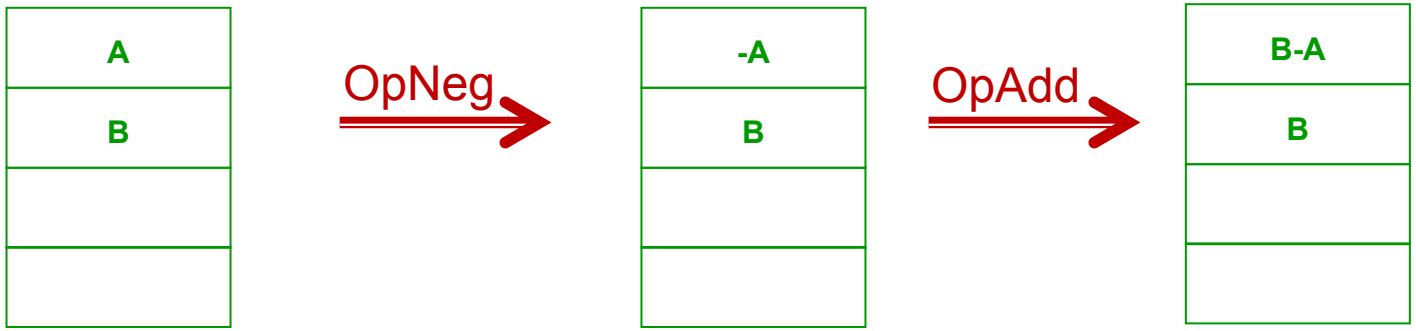
负数情况，反转结果

基于栈的乘法运算程序(4)

```
        LD R1,OpMult_Save1
        LD R2,OpMult_Save2
        LD R3,OpMult_Save3
        LD R5,OpMult_Save5
        LD R7,OpMult_Save7
        RET
OpMult_Save0 .BLKW #1
OpMult_Save1 .BLKW #1
OpMult_Save2 .BLKW #1
OpMult_Save3 .BLKW #1
OpMult_Save5 .BLKW #1
OpMult_Save7 .BLKW #1
```

减法实现

1. 将栈顶数据（A）转换为其负数（–A）
2. 再调用加法程序（B–A）
3. 即可实现减法



基于栈的负数转换运算程序

```
; Subroutine to pop the top of the stack, form its negative,
; and push the result onto the stack.
OpNeg ST R0,OpNeg_Save0
      ST R5,OpNeg_Save5
      ST R7,OpNeg_Save7
      JSR POP ; Get the source operand.
      ADD R5,R5,#0 ; Test for successful pop
      BRp OpNeg_Exit ; Branch if failure.
      NOT R0,R0
      ADD R0,R0,#1 ; Form the negative of source.
      JSR PUSH ; Push result onto the stack.
OpNeg_Exit LD R0,OpNeg_Save0
      LD R5,OpNeg_Save5
      LD R7,OpNeg_Save7
      RET
OpNeg_Save0 .BLKW #1
OpNeg_Save5 .BLKW #1
OpNeg_Save7 .BLKW #1
```

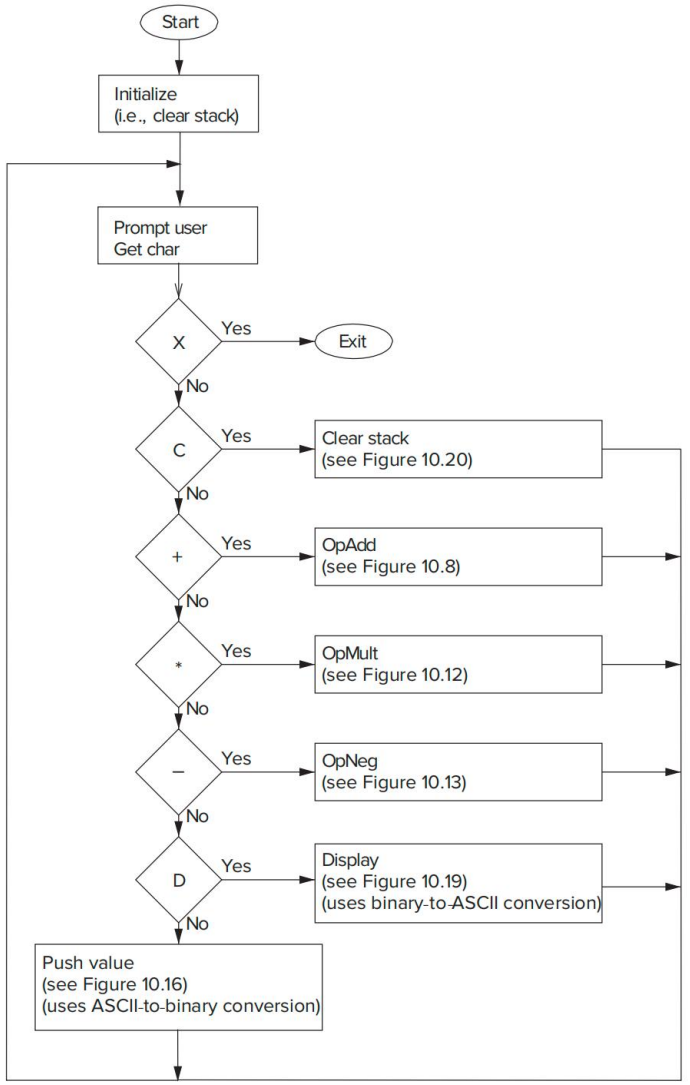
检查POP是否成功

将栈顶数据修改为其负数

计算器（The Calculator）

- 计算器工作流程

- X Exit the simulation.
- D Display the value at the top of the stack.
- C Clear all values from the stack.
- + Pop the top two elements A,B off the stack and push A+B.
- * Pop the top two elements A,B off the stack and push A*B.
- — Pop the top element A off the stack and push “minus” A.
- Enter or LF Push the value typed on the keyboard onto the top of the stack.



计算器使用方法

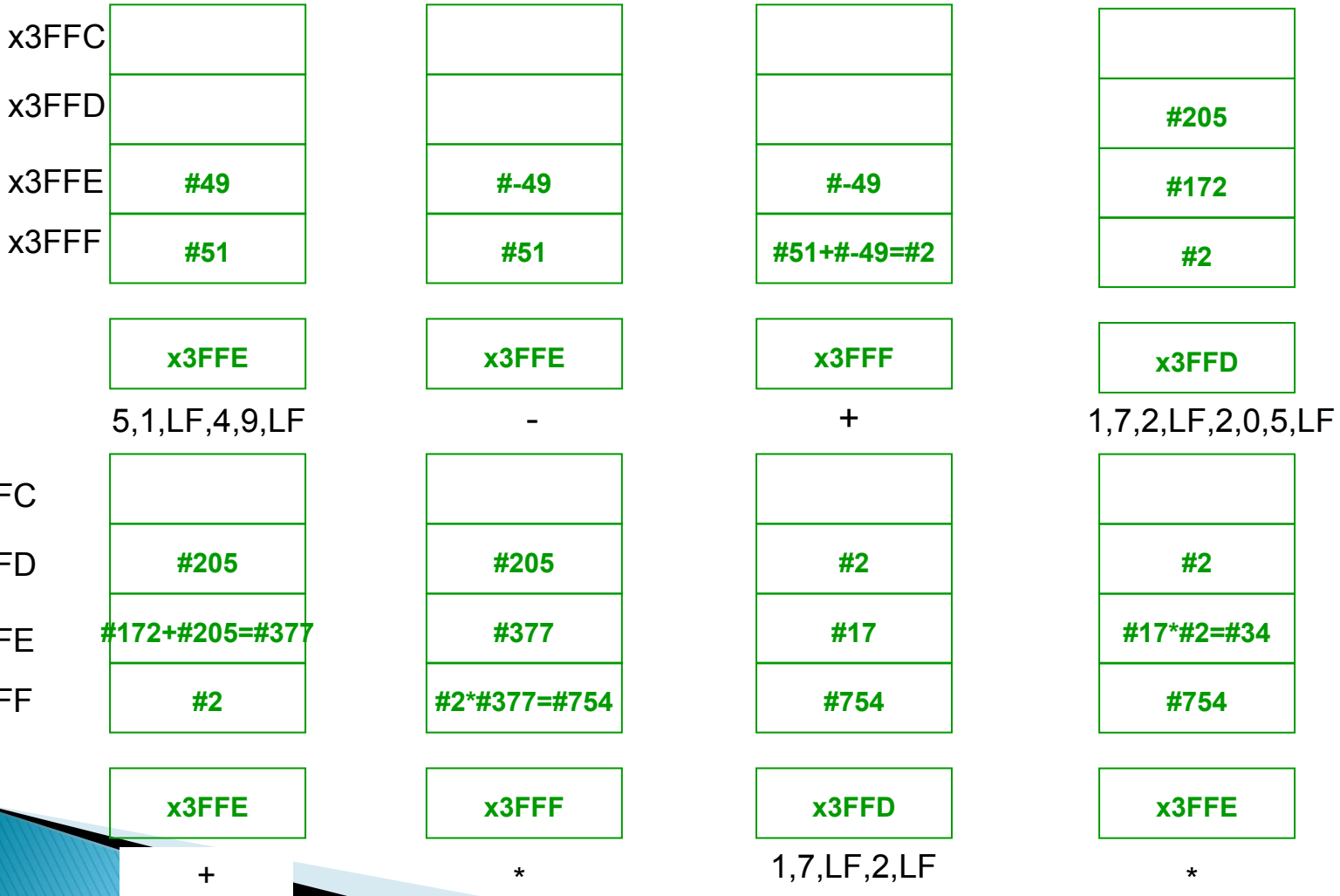
- For example, to calculate
  - (51 – 49) * (172 + 205) – (17 * 2)

- Display the result 720 on the monitor, one types the following sequence of keys on the keyboard:

- 5,1,LF,4,9,LF,—,+,1,7,2,LF,2,0,5,LF,+,*,1,7,LF,2,LF,*,—,+,D.

计算器的栈内变化

- 5,1,LF,4,9,LF,−,+,1,7,2,LF,2,0,5,LF,+,*,1,7,LF,2,LF,*,−,+,D.
- 计算器的栈内变化：



计算器的栈内变化（2）

- 5,1,LF,4,9,LF,—,+,1,7,2,LF,2,0,5,LF,+,*,1,7,LF,2,LF,*,—,+,D.
- 计算器的栈内变化：